Objetivos y requisitos funcionales del Telegram Bot

Proyecto académico de bot con portal web, verificación e integraciones externas

Descripción general

El proyecto Telegram Bot consiste en una aplicación que permite a los usuarios registrarse desde una página web, verificar su cuenta de Telegram, autorizar aplicaciones externas y usar un bot conversacional desde Telegram. El sistema integra funciones de inteligencia artificial, juegos, utilidades, calendario, consulta de repositorios, información de YouTube y almacenamiento en una base de datos Neon PostgreSQL. El despliegue se plantea en Railway para mantener la página y el bot disponibles en línea.

Los documentos anexos del profesor fueron tratados únicamente como referencia de estructura académica. El contenido de este documento corresponde al proyecto Telegram Bot.

Objetivo general

Desarrollar un sistema de bot para Telegram con portal web de verificación y autorización de aplicaciones, que permita a los usuarios interactuar con funciones de inteligencia artificial, automatización, consulta de información y servicios externos como Google Calendar, GitHub y YouTube, utilizando una base de datos en la nube para conservar la información necesaria del sistema.

Objetivos específicos

- Implementar una página web inicial donde el usuario pueda registrarse, verificar su cuenta de Telegram y acceder al bot.
- Permitir la conexión segura de aplicaciones externas mediante permisos otorgados por el usuario.
- Desarrollar comandos básicos de Telegram para menú, ayuda, juegos, utilidades y conversación con inteligencia artificial.
- Integrar Google Calendar para consultar eventos y programar actividades desde el chat del bot.
- Integrar GitHub para consultar repositorios, issues, pull requests, commits recientes o cambios pendientes.
- Integrar YouTube para buscar videos, consultar información básica y responder solicitudes relacionadas con contenido audiovisual.
- Almacenar usuarios, preferencias, permisos, historial resumido y registros del sistema en Neon PostgreSQL.
- Desplegar el sistema en Railway usando variables de entorno para proteger tokens y claves de acceso.
- Definir controles administrativos para configurar servicios, revisar errores y mantener el funcionamiento del bot.

Módulos del sistema

El sistema se divide en módulos para separar responsabilidades y facilitar el desarrollo incremental.

Módulo de usuario o cliente

Gestiona la interacción del usuario final con la página web y con el bot de Telegram.

Código	Requisito funcional	Actor	Prioridad
RF-01	El sistema debe permitir que el usuario acceda a la página web principal del proyecto. Dentro del detalle se registraar los datos de username	Usuario	Alta
RF-02	El sistema debe permitir que el usuario verifique su identidad de Telegram mediante inicio de sesión o enlace de verificación.	Usuario	Alta
RF-03	El sistema debe mostrar al usuario el estado de sus aplicaciones conectadas.	Usuario	Media
RF-04	El sistema debe permitir que el usuario abra el chat del bot desde la página web.	Usuario	Alta
RF-05	El sistema debe permitir que el usuario consulte ayuda o menú de funcionalidades desde Telegram.	Usuario	Alta

Módulo de administrador

Permite al responsable del proyecto configurar, controlar y revisar el funcionamiento del sistema.

Código	Requisito funcional	Actor	Prioridad
RF-06	El sistema debe validar que solo usuarios autorizados puedan acceder a funciones administrativas.	Administrador	Alta
RF-07	El administrador debe poder configurar tokens, claves API y variables necesarias para las integraciones.	Administrador	Alta
RF-08	El administrador debe poder revisar registros básicos de errores y actividad del bot.	Administrador	Media
RF-09	El administrador debe poder activar o desactivar integraciones según el estado del proyecto.	Administrador	Media
RF-10	El administrador debe poder definir repositorios o servicios predeterminados para las consultas del bot.	Administrador	Media

Módulo de portal web

Funciona como entrada principal para registro, verificación, permisos e inicio del flujo hacia Telegram.

Código	Requisito funcional	Actor	Prioridad
RF-11	La página web debe mostrar una interfaz inicial con información básica del proyecto y acceso a verificación.	Usuario	Alta
RF-12	La página debe registrar el identificador de Telegram del usuario verificado.	Usuario	Alta
RF-13	La página debe ofrecer botones para conectar Google Calendar, GitHub y YouTube cuando estén disponibles.	Usuario	Alta
RF-14	La página debe guardar el estado de conexión de cada aplicación externa.	Sistema	Alta
RF-15	La página debe redirigir al usuario hacia el bot de Telegram cuando finalice el registro.	Usuario	Alta

Módulo del bot de Telegram

Procesa mensajes, comandos y respuestas dentro del chat de Telegram.

Código	Requisito funcional	Actor	Prioridad
RF-16	El bot debe responder al comando /start con un saludo y orientación inicial.	Usuario	Alta
RF-17	El bot debe responder a /menu, /ayuda o /help con las funcionalidades disponibles.	Usuario	Alta
RF-18	El bot debe validar si el usuario está registrado antes de permitir funciones conectadas.	Sistema	Alta
RF-19	El bot debe ejecutar comandos de juegos y utilidades como trivia, dado, moneda, hora, eco y piedra papel tijera. Mencionar detalle de cda juego, tiempo, etc.	Usuario	Media
RF-20	El bot debe manejar errores de comandos mostrando mensajes claros de uso correcto.	Usuario	Alta

Módulo de inteligencia artificial

Permite conversaciones naturales y apoyo textual mediante un modelo de IA.

Código	Requisito funcional	Actor	Prioridad
RF-21	El sistema debe enviar mensajes libres del usuario a un servicio de IA cuando no correspondan a comandos.	Usuario	Alta
RF-22	El sistema debe conservar un historial reciente de conversación por chat.	Sistema	Media
RF-23	El sistema debe responder en español de forma breve y comprensible.	Usuario	Media
RF-24	El sistema debe permitir borrar la memoria de conversación con un comando.	Usuario	Media
RF-25	El sistema debe mostrar una respuesta de error controlada cuando la API de IA no esté disponible.	Sistema	Alta

Módulo de Google Calendar

Permite consultar y crear eventos de calendario desde el bot.

Código	Requisito funcional	Actor	Prioridad
RF-26	El sistema debe permitir conectar una cuenta de Google Calendar desde la página web.	Usuario	Alta
RF-28	El bot debe consultar la agenda del usuario por día, semana o fecha específica.	Usuario	Alta
		Usuario	Alta
RF-30	El sistema debe informar cuando el usuario no tenga Calendar conectado.	Sistema	Alta

Módulo de GitHub

Permite consultar información de repositorios y cambios pendientes.

Código	Requisito funcional	Actor	Prioridad
RF-31	El sistema debe permitir conectar GitHub desde la página web para acceder a repositorios autorizados.	Usuario	Alta

RF-32	El bot debe consultar y mostrar los issues abiertos de un repositorio indicado, incluyendo título, estado, fecha y enlace, para que el usuario identifique tareas, errores o solicitudes pendientes dentro del proyecto.	Usuario	Media
RF-33	El bot debe consultar los pull requests pendientes de revisión o fusión, mostrando título, autor, rama, estado y enlace, para que el usuario sepa qué cambios de código están esperando aprobación antes de integrarse al repositorio principal.	Usuario	Media
RF-34	El bot debe mostrar commits recientes o actividad básica del repositorio.	Usuario	Media
RF-35	El sistema debe informar cuando no tenga permisos para consultar un repositorio privado.	Sistema	Alta

Nota sobre GitHub: un issue representa una tarea, error o solicitud registrada en el repositorio; un pull request representa una propuesta de cambio en el código que debe revisarse antes de fusionarse con la rama principal.

Módulo de YouTube

Permite buscar y consultar información básica de videos o canales.

Código	Requisito funcional	Actor	Prioridad
RF-36	El sistema debe permitir configurar una clave de YouTube Data API para consultas.	Administrador	Media
RF-37	El bot debe reconocer enlaces de YouTube enviados por el usuario.	Usuario	Media
RF-38	El bot debe buscar videos por palabras clave cuando el usuario lo solicite.	Usuario	Media
RF-39	El bot debe mostrar título, canal y enlace del resultado encontrado.	Usuario	Media
RF-40	El sistema debe informar cuando se agote la cuota o falle la consulta de YouTube.	Sistema	Alta

Módulo de base de datos Neon PostgreSQL

Almacena la información necesaria para mantener continuidad y permisos del sistema.

Código	Requisito funcional	Actor	Prioridad
RF-41	El sistema debe guardar usuarios verificados con su identificador de Telegram.	Sistema	Alta
RF-42	El sistema debe guardar el estado de las integraciones autorizadas por cada usuario.	Sistema	Alta
RF-43	El sistema debe guardar preferencias y memoria resumida cuando el usuario lo permita.	Sistema	Media
RF-44	El sistema debe guardar logs básicos para diagnóstico del administrador.	Administrador	Media
RF-45	El sistema debe manejar fallos de base de datos sin detener completamente el bot.	Sistema	Alta

Módulo de despliegue y operación

Mantiene el sistema disponible en Railway con configuración protegida.

Código	Requisito funcional	Actor	Prioridad
RF-46	El sistema debe leer tokens y claves desde variables de entorno, no desde el código fuente.	Administrador	Alta
RF-47	El sistema debe iniciar el bot automáticamente al desplegarse en Railway.	Sistema	Alta
RF-48	El sistema debe exponer la página web pública para registro y autorización.	Usuario	Alta
RF-49	El sistema debe mostrar logs de arranque y errores para diagnóstico.	Administrador	Media
RF-50	El sistema debe permitir cambiar configuración sin modificar directamente el código cuando sea posible.	Administrador	Media

Funcionalidades principales esperadas

- Registro y verificación de usuario desde una página web.
- Autorización de aplicaciones externas conectadas al bot.
- Interacción mediante comandos en Telegram.
- Conversación con inteligencia artificial y memoria de contexto.
- Juegos y utilidades básicas para interacción inicial.

- Creación y consulta de eventos en Google Calendar.
- Consulta de cambios pendientes en GitHub.
- Búsqueda o consulta de información de YouTube.
- Persistencia de usuarios, permisos y preferencias en Neon PostgreSQL.
- Despliegue en Railway con configuración por variables de entorno.

Alcance recomendado para la primera entrega

Para una primera versión académica se recomienda priorizar portal web, verificación de Telegram, menú del bot, IA, juegos/utilidades, Neon y Railway. Calendar, GitHub y YouTube pueden agregarse progresivamente.

Información almacenada en la base de datos

El sistema utilizará una base de datos Neon PostgreSQL para guardar la información necesaria del portal web, los usuarios verificados, las autorizaciones de aplicaciones y el funcionamiento del bot. No se deben guardar contraseñas de Telegram ni claves visibles en texto plano; los tokens de acceso deben almacenarse de forma segura y usarse únicamente para ejecutar las funciones autorizadas por el usuario.

Tabla / entidad	Información que se guarda	Uso dentro del sistema	Observación
usuarios	Identificador interno, id de Telegram, nombre de usuario de Telegram, nombre visible, fecha de registro, estado de verificación y fecha de última actividad.	Permite reconocer al usuario cuando entra a la página o escribe al bot.	Dato principal del sistema.
integraciones	Usuario asociado, aplicación conectada, estado de conexión, fecha de autorización, fecha de expiración del token y permisos concedidos.	Permite saber si el usuario conectó Calendar, GitHub o YouTube.	No guardar permisos no aceptados.
tokens	Usuario asociado, servicio externo, token cifrado, refresh token cifrado cuando aplique y fecha de actualización.	Permite ejecutar acciones autorizadas en servicios externos.	Debe protegerse con cifrado.
preferencias	Idioma, zona horaria, repositorio predeterminado, calendario predeterminado, preferencias de respuesta y opciones de privacidad.	Personaliza las respuestas y evita pedir los mismos datos varias veces.	El usuario debe poder cambiarlas.
memoria	Usuario, chat de Telegram, resumen de conversación, últimos temas tratados y fecha de actualización.	Permite que la IA conserve contexto entre conversaciones.	Guardar resumen, no todo innecesariamente .

eventos	Usuario, id del evento externo, título, fecha, hora, descripción, estado y enlace del evento.	Lleva control de eventos creados desde el bot.	La fuente principal sigue siendo Google Calendar.
github	Usuario, repositorio consultado, tipo de consulta, resultado resumido, fecha y estado de la consulta.	Permite historial de revisiones de issues, pull requests o commits.	Útil para auditoría y caché.
comandos	Usuario, chat, comando ejecutado, parámetros principales, fecha, resultado y mensaje de error si ocurrió.	Sirve para estadísticas, diagnóstico y mejora del bot.	No guardar contenido sensible.
puntajes	Usuario, juego, puntaje acumulado, partidas jugadas, aciertos y última participación.	Conserva puntuaciones de trivia u otros juegos.	Opcional para primera versión.
logs	Tipo de evento, módulo, descripción del error, fecha, usuario relacionado y nivel de severidad.	Ayuda al administrador a revisar fallos del portal, bot o integraciones.	No incluir tokens ni claves.
configuracion	Nombre del bot, URL pública, servicios activos, límites de uso, repositorios por defecto y parámetros generales.	Permite administrar el sistema sin modificar código constantemente.	Acceso solo administrador.

Relación entre módulos y datos guardados

- Módulo de usuario o cliente: guarda datos de registro, verificación, preferencias y estado de conexión con aplicaciones externas.
- Módulo de administrador: guarda configuración general, servicios activos y logs de funcionamiento.
- Módulo de portal web: guarda usuarios verificados, sesiones de autorización y aplicaciones conectadas.
- Módulo del bot de Telegram: guarda comandos ejecutados, memoria resumida, puntajes y actividad reciente.
- Módulos externos: Calendar, GitHub y YouTube guardan identificadores, resultados resumidos y estado de consultas para trazabilidad y ahorro de cuota.

Reglas de almacenamiento

- El sistema debe guardar únicamente información necesaria para el funcionamiento del bot.
- Los tokens de acceso de aplicaciones externas deben almacenarse cifrados o protegidos.
- Las claves como BOT_TOKEN, GROQ_API_KEY y DATABASE_URL deben permanecer en variables de entorno de Railway.
- La memoria de IA debe guardarse como resumen para reducir datos innecesarios.
- Los registros de errores no deben incluir contraseñas, tokens ni información sensible.

Requisito funcional agregado

Código	Requisito funcional	Actor	Prioridad
RF-51	El sistema debe registrar en Neon PostgreSQL la información de usuarios, permisos, preferencias, historial resumido, consultas, eventos, comandos, puntajes y logs necesarios para el funcionamiento del Telegram Bot.	Sistema / Administrador	Alta